

QR을 찍어 직접 BMS시스템을 제어해보세요! --□

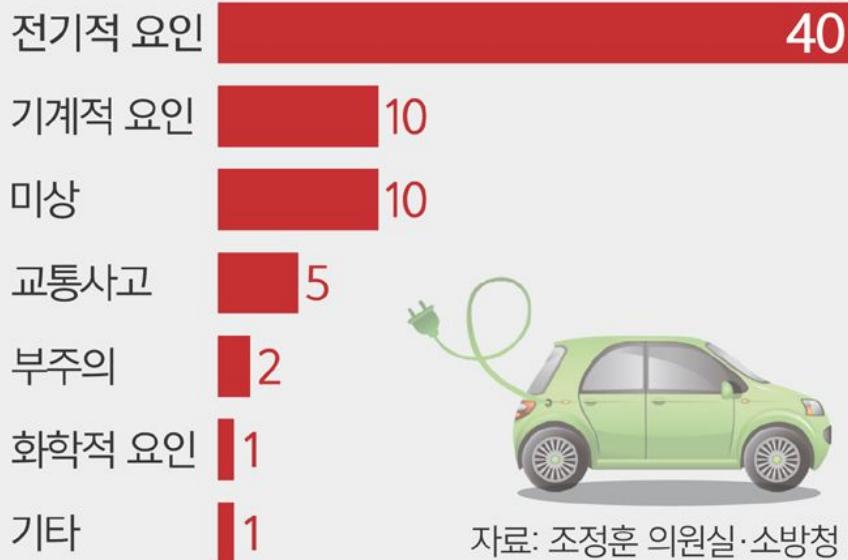


BMS시스템

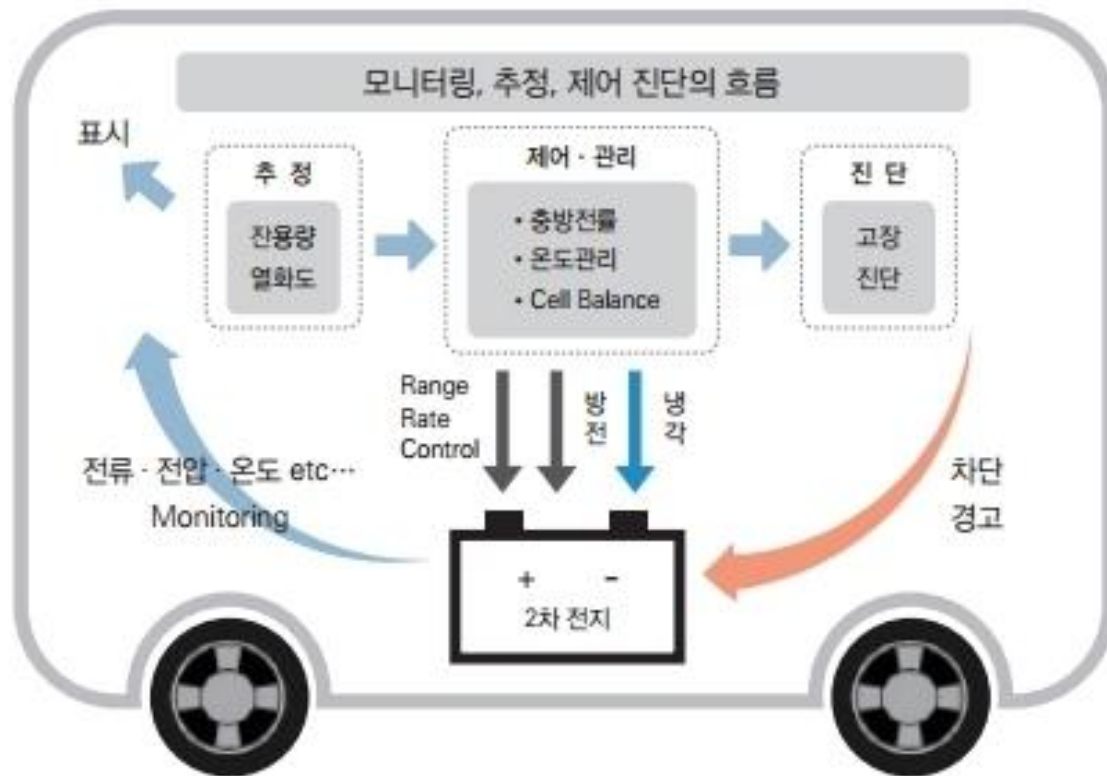
시스템/SW 폰 노이만 팀

프로젝트 설계 배경

화재원인(단위: 건)



- 전기차 화재 심각성(회사와 소비자의 피해금액, 인명피해)



HOW TO
SOLVE
THIS
PROBLEM
?

BMS!

◇ BMS 구성과 작동 흐름도 [사진=KISTI]

전기자동차 **BMS**에 배터리의 상태를 파악하고 이상상황시 이를 즉각적으로 알리고 자동적으로 배터리를 자동차로부터 전기적 분리시켜 대처

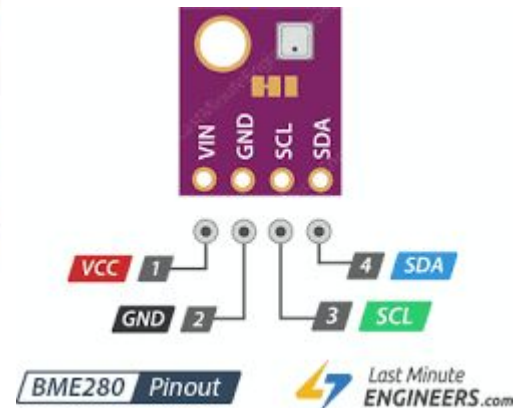
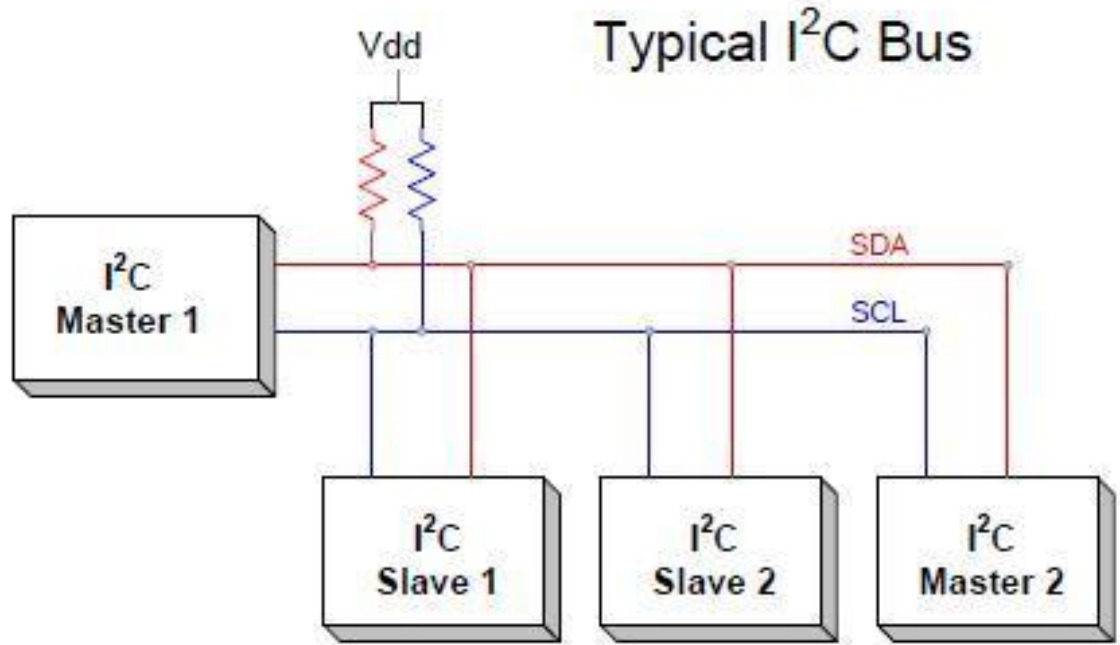
프로젝트 목표

- 배터리 관리 시스템(BMS) 모니터링 및 제어
- 실시간 데이터 수집 및 시각화
- 모터 제어 기능 추가 (on/off, 속도 조정)
- MQTT 프로토콜/i2c/can 을 활용한 통신

WHAT WE NEED?

- 전류,전압,온/습도센서
- 자동차 기어 구현.(drive, rear, stop)
- 배터리 차단 회로
- 저전력 기능
- 원격 제어 시스템
- 회로설계 및 작동원리 이해능력
- 실생활 적용

전류, 전압온/습 도센서 (i2C)



Slave1

Slave2

자동차
기어 구현.(
drive, rear,
stop)



자동차 기어 구현.(drive)

```
else if(((D == 1) && (E == 0))){  
    if (aspeed >= 40){ //전진모드여도  
        가속을 안하면 정지할수 있도록
```

```
        digitalWrite(25,HIGH);  
        digitalWrite(12,LOW);  
        analogWrite(26,aspeed);  
        digitalWrite(14,LOW);  
        // digitalWrite(10,LOW);
```

```
    }
```

자동차 기어 구현.(rear)

```
else if( (R == 1) && (E == 0)){  
    if (aspeed >= 40){
```

```
        digitalWrite(25, LOW);  
        digitalWrite(12, HIGH);  
        analogWrite(26, aspeed);  
        digitalWrite(14, LOW);  
        //digitalWrite(10, LOW);
```

```
    }
```

자동차
기어
구현.(stop
)

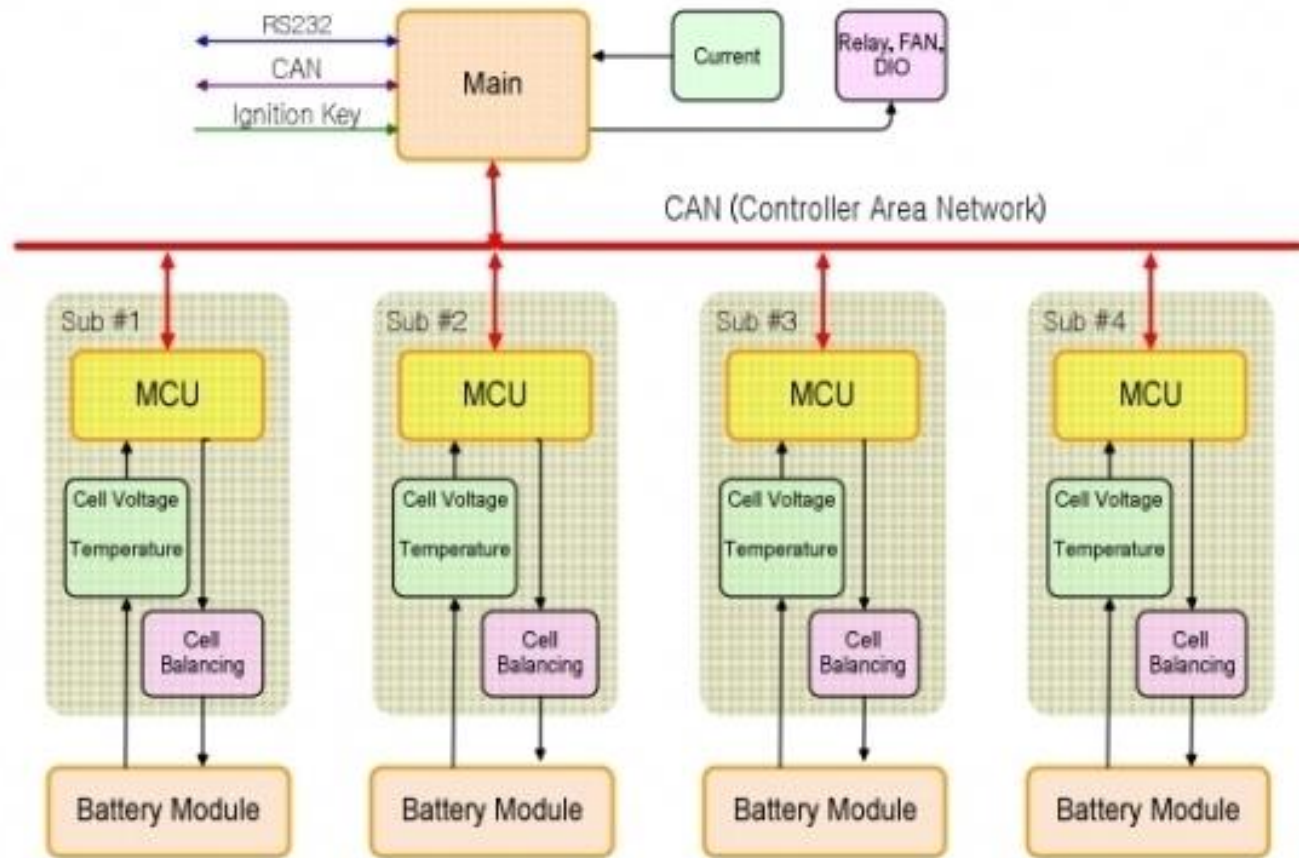
```
//정지  
else if( (P == 1) && (E == 0)){  
  
    digitalWrite(25,LOW);  
    digitalWrite(12,LOW);  
    digitalWrite(14,LOW);  
    //digitalWrite(10,LOW);  
    analogWrite(26,0);  
}
```

배터리 전원
원천 차단

원격제어 시스템
{BMS/power}
On Off

- 릴레이 모듈

WHAT WE NEED?

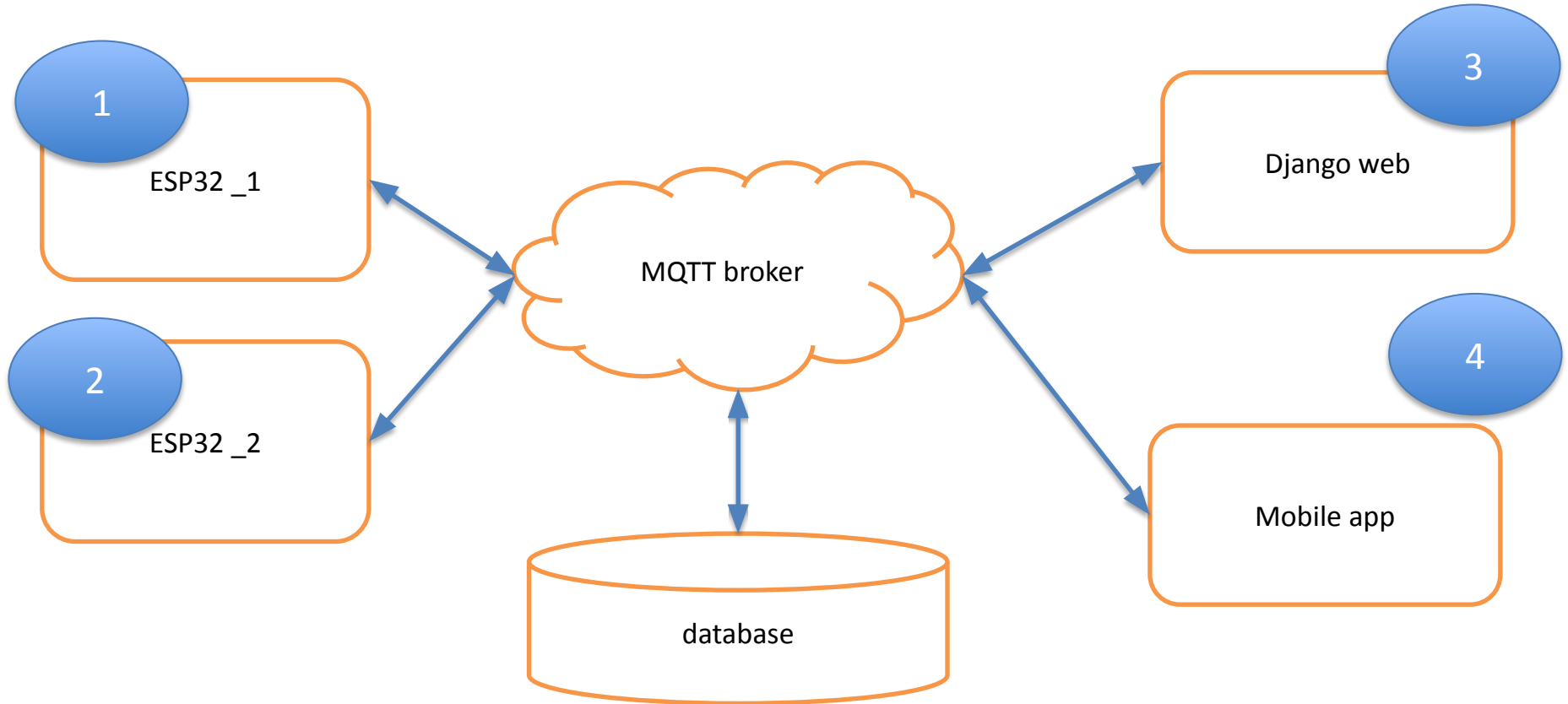


◇ BMS 및 배터리 팩 구성도 [사진=파워로직스]

시스템 아키텍처

- IoT Device (ESP32) → MQTT Broker → Django 서버 → 사용자 인터페이스
- IoT Device: 센서 데이터 수집
- MQTT Broker: 메시지 전송 및 수신
- Django Web Application: 데이터 저장, 시각화, 모터 제어
- User Interface: 웹 기반 실시간 모니터링 및 제어

전체 구조



1

ESP32_1

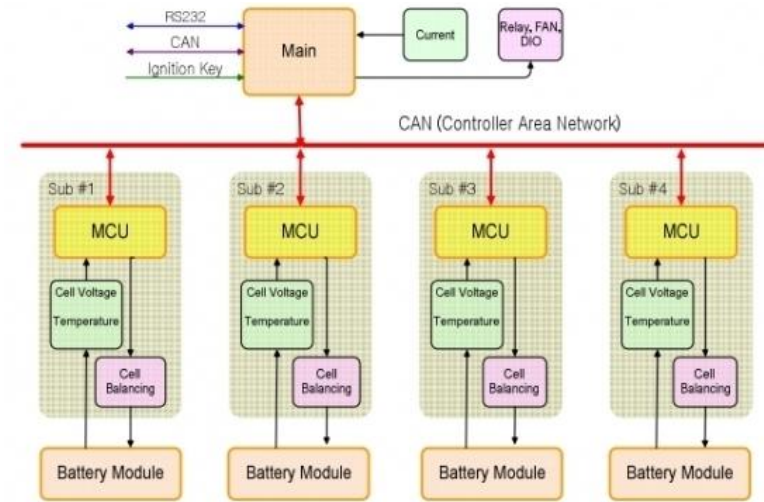
자동차 모터

CAN 통신모듈

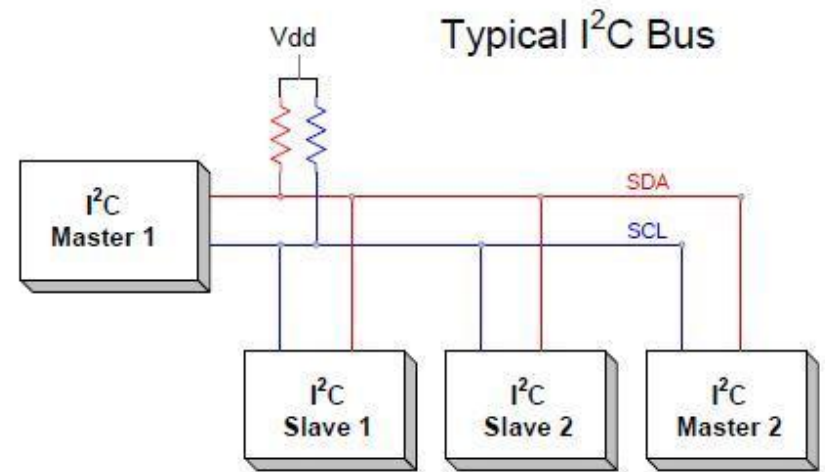
ESP32_1

전류전압센서

온도습도센서

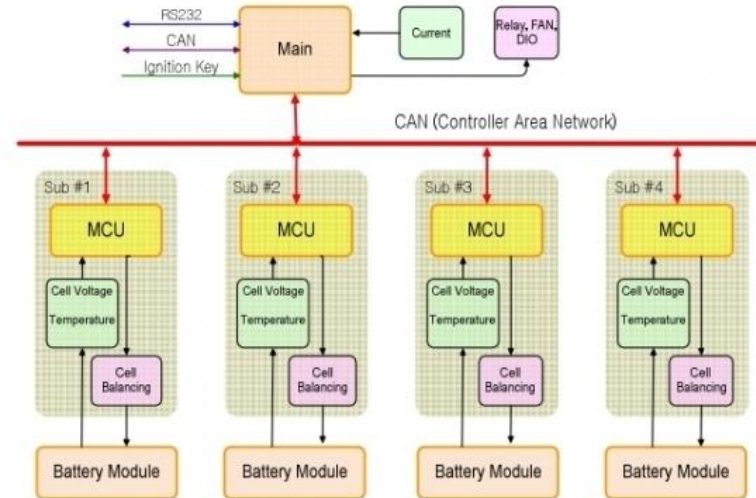


◇ BMS 및 배터리 팩 구성도 [사진=파워로직스]



2

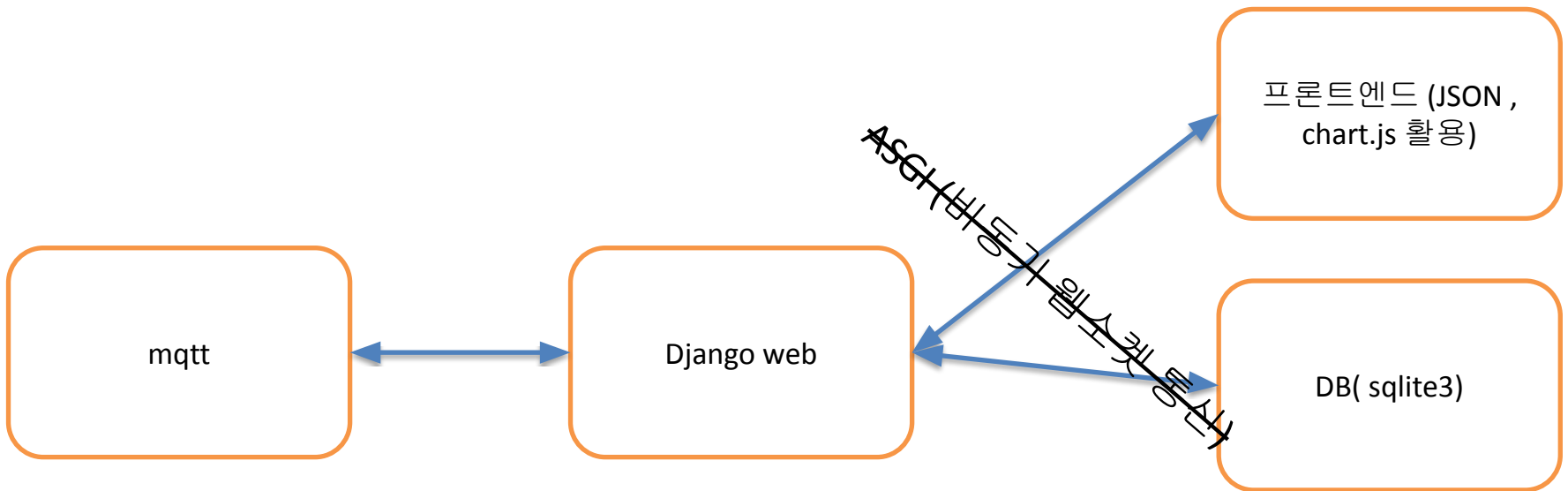
ESP32_2

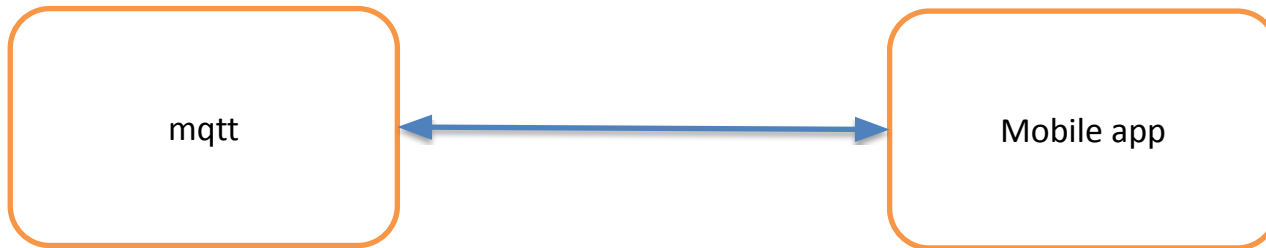


◇ BMS 및 배터리 팩 구성도 [사진=파워로직스]

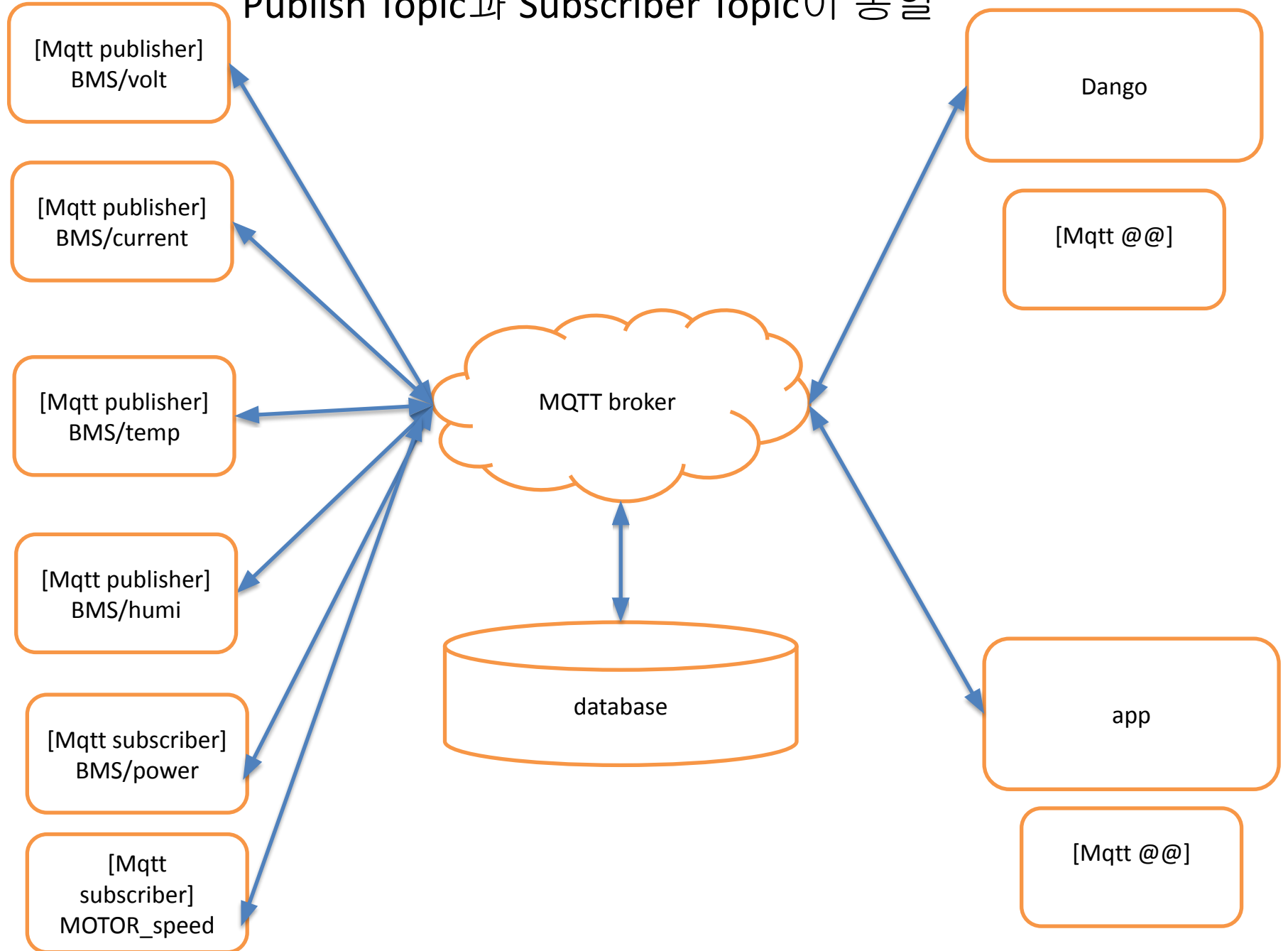
ESP32_2

CAN 통신모듈





Publish Topic과 Subscriber Topic이 동일



DB구조

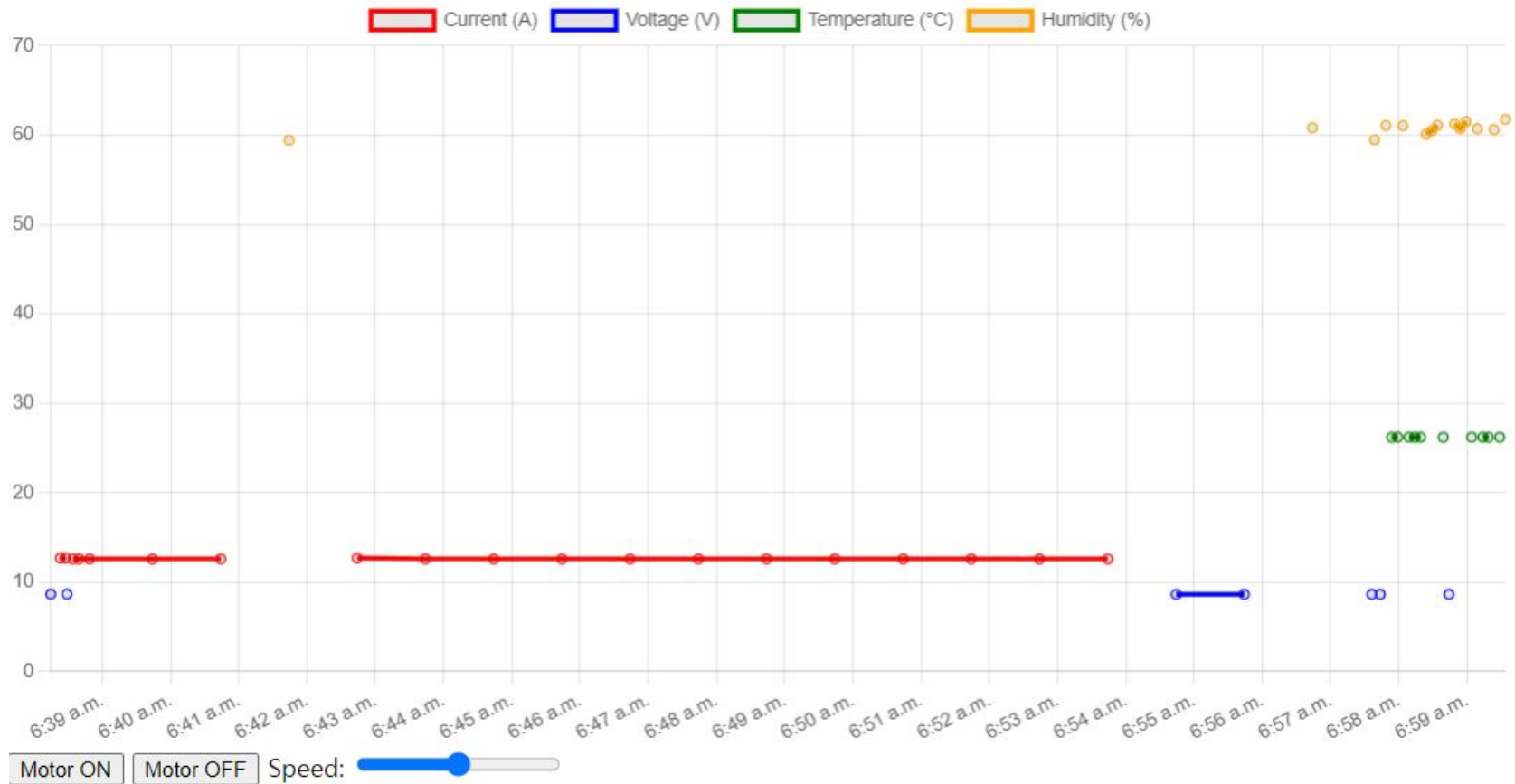
```
class SensorData(models.Model):
    current = models.FloatField(null=True, blank=True)
    voltage = models.FloatField(null=True, blank=True)
    temperature = models.FloatField(null=True, blank=True)
    humidity = models.FloatField(null=True, blank=True)
    timestamp = models.DateTimeField(auto_now_add=True)

    def __str__(self):
        return f"Current: {self.current}, Voltage: {self.voltage}, Temp: {self.temperature}, Humidity: {self.humidity}"

class MotorControl(models.Model):
    timestamp = models.DateTimeField(auto_now_add=True)
    power = models.IntegerField() # 0: OFF, 1: ON
    speed = models.FloatField() # 모터 속도

    def __str__(self):
        return f"Motor Control at {self.timestamp} - Power: {'ON' if self.power else 'OFF'}, Speed: {self.speed}"
```

프로젝트 결과물 _ 웹 사이트



Sensor Data

현장 시연

QR을 찍어 직접 BMS시스템을
제어해보세요! --□



송실 반도체의 날에도 이와 같이
실제 기기를 방문자가 체험할 수 있는 것을 의도